Федеральное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет Программной инженерии и компьютерной техники

Дисциплина: «Основы профессиональной деятельности»

Лабораторная работа №5
«Асинхронный обмен данными с ВУ»
Вариант **3134**

Выполнил:
Крамарь Михаил Романович
P3113

Проверила:
Остапенко Ольга Денисовна

г. Санкт-Петербург
2026

Содержание

Текст задания 3
Ход работы 4
Назначение программы: 6
ОПИ и ОДЗ: 6
Таблица трассировки 8
Дополнительное задание 10
Вывод 15

Текст задания

По выданному преподавателем варианту разработать программу асинхронного обмена данными с внешним устройством. При помощи программы осуществить ввод или вывод информации, используя в качестве подтверждения данных сигнал (кнопку) готовности ВУ.

1. Программа осуществляет асинхронный ввод данных с ВУ-2
2. Программа начинается с адреса 46E16. Размещаемая строка находится по адресу 5BD16.
3. Строка должна быть представлена в кодировке КОИ-8.
4. Формат представления строки в памяти: АДР1: СИМВ1 СИМВ2 АДР2: СИМВ3 СИМВ4 ... СТОП_СИМВ.
5. Ввод или вывод строки должен быть завершен по символу с кодом 0D (CR). Стоп символ является обычным символом строки и подчиняется тем же правилам расположения в памяти что и другие символы строки.

Ход работы

Описание программы:

Передаваемое сообщение: «Миша»

В кодировке ***КОИ-8***: 0xED 0xC9 0xDB 0xC1

В кодировке ***UTF-8***: 0xD0 0x9C 0xD0 0xB8 0xD1 0x88 0xD0 0xB0

В кодировке ***UTF-16***: 0x04 0x1C 0x04 0x38 0x04 0x48 0x04 0x30

Текст программы на ассемблере:

```
ORG 0x46E ; Адрес начала нашей программы

START:  CLA ; Очищаем аккумулятор
S1:  IN 5 ; Ожидание ввода первого символа, заносим SR в AC
     AND #0x40 ; Проверяем 6-ой бит SR (битовой маской)
     BEQ S1 ; Если флаг Z==1 - spin-loop (переходим к метке S1)
     IN 4 ; Ввод первого символа

     CMP FINISH ; выставляем флаги по результату AC - FINISH
     BEQ EXIT; если Z==1, переходим к выходу

     SWAB ; Переносим первый символ в старшую часть AC
     ST (RES) ; Сохраняем в ячейку 0x5BD

S2:  IN 5 ; Ожидание ввода второго символа
     AND #0x40 ;
     BEQ S2 ;

     CLA ; Очищаем аккумулятор

     IN 4 ; Ввод второго символа

     CMP FINISH
     BEQ EXIT_PAIR ; Если получили стоп-символ, идём склеивать с первой буквой

     OR (RES) ; Добавляем первый введённый символ на место старшего байта
     ST (RES)+ ; Сохраняем слово из двух букв в ячейку RES, переходим к следующей за ней ячейкой
     CLA ; Очищаем аккумулятор на будущее
     JUMP S1 ; Переходим ко вводу следующего (нечётного) символа

EXIT_PAIR: OR (RES) ; Доклеиваем стоп-символ, если попали сюда
```

```
EXIT:   ST (RES) ; Загружаем всё в RES
        HLT

RES:    WORD 0x5BD ; Адрес хранения результата
FINISH: WORD 0x0D ; Константа - стоп символ
```

Текст исходной программы

Адрес	*Код команды*	*Мнемоника*	*Комментарии*
46E	**+0200**	**CLA**	**Очистка аккумулятора: 0 -> AC**
46F	**1205**	**IN 5**	**Чтение регистра состояния SR ВУ-2 в AC Метка: S1**
470	**2F40**	**AND #0x40**	**Логическое «&» с непосредственной загрузкой операнда AC & 0040 -> AC**
471	**F0FD**	**BEQ S1**	**Если Z==1, переход к адресу с меткой S1**
472	**1204**	**IN 4**	**Чтение регистра данных DR ВУ-2 в AC**
473	**7E12**	**CMP FINISH**	**Установить флаги по результату операции: AC - 000D**
474	**F00E**	**BEQ EXIT**	**Если Z==1, переход к адресу с меткой EXIT**
475	**0680**	**SWAB**	**Обмен старшего и младшего байта аккумулятора**
476	**E80E**	**ST (RES)**	**Косвенное относительное сохранение в ячейку с адресом из RES**
477	**1205**	**IN 5**	**Чтение регистра состояния SR ВУ-2 в AC Метка: S2**
478	**2F40**	**AND #0x40**	**Логическое «&» с непосредственной загрузкой операнда AC & 0040 -> AC**
479	**F0FD**	**BEQ S2**	**Если Z==1, переход к адресу с меткой S2**

47A	**0200**	**CLA**	**Очистка аккумулятора: 0 -> AC**
47B	**1204**	**IN 4**	**Чтение регистра данных DR ВУ-2 в AC**
47C	**7E09**	**CMP FINISH**	**Установить флаги по результату операции: AC - 000D**
47D	**F004**	**BEQ EXIT_PAIR**	**Если Z==1, переход к адресу с меткой EXIT_PAIR**
47E	**3806**	**OR (RES)**	**Косвенное относительное логическое «или»: AC or (RES) -> AC**
47F	**EA05**	**ST (RES)+**	**Косвенное автоинкрементное сохранение аккумулятора в ячейку с адресом из RES**
480	**0200**	**CLA**	**Очистка аккумулятора: 0 -> AC**
481	**CEED**	**JUMP S1**	**Прямой относительный прыжок к адресу с меткой S1**
482	**3802**	**OR (RES)**	**Косвенное относительное логическое «или»: AC or (RES) -> AC**
483	**E801**	**ST (RES)**	**Косвенное относительное сохранение в ячейку с адресом из RES**
484	**0100**	**HLT**	**Останов. Переход в пультовый режим**
485	05BD	-	Адрес хранения результата - RES
486	000D	-	Константа: стоп-слово - FINISH

Назначение программы:

- Данная программа осуществляет посимвольный асинхронный ввод данных с ВУ-2, посимвольно записывает их в память. Программа получает символы до тех пор, пока не будет введён стоп-символ при помощи ВУ-2 с кодировкой 0x0D, который она запишет в память и прекратит своё выполнение.

ОПИ и ОДЗ:

- ***Область представления***:

- RES - 11-ти разрядная ячейка со ссылкой на результат (это адрес)
- FINISH - 16-ти разрядная константа
- 0x5BD - ? - 16-ти разрядные ячейки, которые хранят по два символа в кодировке КОИ-8!

• ***Область допустимых значений:***
  - RES (адрес каждой ячейки «массива», который хранит результат ввода пользователя): [0x485; 0x7FF]
  - Каждый введённый символ: [0x00; 0xFF]

• **Расположение данных и программы в памяти ЭВМ**

Основная программа:

- Команды: ***46E - 484;***
- Исходные данные: RES - ***485***, FINISH - ***486***
- Итоговый результат: ***5BD - 7FF***;

• **Адреса первой и последней исполняемых команд**

Основная программа:

- Первая команда: ***46E***;
- Последняя команда: ***484***;

Таблица трассировки

Выполняемая команда		Содержимое регистров процессора после выполнения команды								Ячейка, содержимое которой изменилось после выполнения команды	
Адрес	Код	IP	CR	AR	DR	SP	BR	AC	NZVC	Адрес	Новый код
46E	0200	46F	0200	46E	0200	000	046E	0000	0100		
46F	1205	470	1205	46F	1205	000	046F	0040	0100		
470	2F40	471	2F40	470	0040	000	0040	0040	0000		
471	F0FD	472	F0FD	471	F0FD	000	0471	0040	0000		
472	1204	473	1204	472	1204	000	0472	00ED	0000		
473	7E12	474	7E12	486	000D	000	0012	00ED	0001		
474	F00E	475	F00E	474	F00E	000	0474	00ED	0001		
475	0680	476	680	475	0680	000	0475	ED00	1001		
476	E80E	477	E80E	5BD	ED00	000	000E	ED00	1001	5BD	ED00
477	1205	478	1205	477	1205	000	0477	ED00	1001		
478	2F40	479	2F40	478	0040	000	0040	0000	0101		
479	F0FD	477	F0FD	479	F0FD	000	FFFD	0000	0101		
477	1205	478	1205	477	1205	000	0477	0040	0101		
478	2F40	479	2F40	478	0040	000	0040	0040	0001		
479	F0FD	47A	F0FD	479	F0FD	000	0479	0040	0001		
47A	0200	47B	0200	47A	0200	000	047A	0000	0101		
47B	1204	47C	1204	47B	1204	000	047B	00C9	0101		
47C	7E09	47D	7E09	486	000D	000	0009	00C9	0001		
47D	F004	47E	F004	47D	F004	000	047D	00C9	0001		
47E	3806	47F	3806	5BD	ED00	000	1236	EDC9	1001		
47F	EA05	480	EA05	5BD	EDC9	000	0005	EDC9	1001	485	05BE
										5BD	EDC9
480	0200	481	0200	480	0200	000	0480	0000	0101		
481	CEED	46F	CEED	481	046F	000	FFED	0000	0101		
46F	1205	470	1205	46F	1205	000	046F	0000	0101		
470	2F40	471	2F40	470	0040	000	0040	0000	0101		
471	F0FD	46F	F0FD	471	F0FD	000	FFFD	0000	0101		

46F	1205	470	1205	46F	1205	000	046F	0040	0101		
470	2F40	471	2F40	470	0040	000	0040	0040	0001		
471	F0FD	472	F0FD	471	F0FD	000	0471	0040	0001		
472	1204	473	1204	472	1204	000	0472	00DB	0001		
473	7E12	474	7E12	486	000D	000	0012	00DB	0001		
474	F00E	475	F00E	474	F00E	000	0474	00DB	0001		
475	0680	476	680	475	0680	000	0475	DB00	1001		
476	E80E	477	E80E	5BE	DB00	000	000E	DB00	1001	5BE	DB00
477	1205	478	1205	477	1205	000	0477	DB00	1001		
478	2F40	479	2F40	478	0040	000	0040	0000	0101		
479	F0FD	477	F0FD	479	F0FD	000	FFFD	0000	0101		
477	1205	478	1205	477	1205	000	0477	0040	0101		
478	2F40	479	2F40	478	0040	000	0040	0040	0001		
479	F0FD	47A	F0FD	479	F0FD	000	0479	0040	0001		
47A	0200	47B	200	47A	0200	000	047A	0000	0101		
47B	1204	47C	1204	47B	1204	000	047B	00C1	0101		
47C	7E09	47D	7E09	486	000D	000	0009	00C1	0001		
47D	F004	47E	F004	47D	F004	000	047D	00C1	0001		
47E	3806	47F	3806	5BE	DB00	000	243E	DBC1	1001		
47F	EA05	480	EA05	5BE	DBC1	000	0005	DBC1	1001	485	05BF
										5BE	DBC1
480	0200	481	0200	480	0200	000	480	0000	0101		
481	CEED	46F	CEED	481	046F	000	FFED	0000	0101		
46F	1205	470	1205	46F	1205	000	046F	0040	0101		
470	2F40	471	2F40	470	0040	000	0040	0040	0001		
471	F0FD	472	F0FD	471	F0FD	000	0471	0040	0001		
472	1204	473	1204	472	1204	000	0472	000D	0001		
473	7E12	474	7E12	486	000D	000	0012	000D	0101		
474	F00E	483	F00E	474	F00E	000	000E	000D	0101		
483	E801	484	E801	5BF	000D	000	0001	000D	0101	5BF	000D
484	0100	485	0100	484	0100	000	0484	000D	0101		

Дополнительное задание

😭😭😭😭😭😭😭*СТРЕЛЬБА ПО ЗАЙЦАМ*😭😭😭😭😭😭😭😭

```
ORG 0x0

; в бэвм походу поломана работа с метками в случае прямой загрузки и при использовании WORD, хотя в методичке указан метод WORD METKA, как рабочий))))


RABBIT:     WORD 0x18, 0x3B, 0x3F, 0x7F, 0xFF, 0xDF, 0xDF, 0x8F, 0x06, 0x00, 0x00
HIT_BOX:    WORD 0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80

HIT_ADR1: WORD $HIT_BOX ; Адрес первого элемента массива HIT_BOX

ADR1:       WORD $RABBIT ; Адрес первого столбца зайца (адрес первого элемента массива)
PTR:  WORD $RABBIT ; Адрес текущего столбца зайца
CNT:  WORD 0xB ; Количество столбцов, необходимых для одного зайца (длина массива)
EXPECTED_FLAG:      WORD 0x20
CURRENT_FLAG:       WORD 0x00
X_INDEX:    WORD 0x00 ; Индекс столбца, в который стреляем
Y_INDEX:    WORD 0x00 ; Индекс строки, в который стреляем

X_COORD: WORD 0x00 ; Ячейка с координатой X
Y_COORD: WORD 0x00 ; Ячейка с координатой Y
DELIMITER:      WORD 0x78 ; Код символа x в KOI8-R
TEMP:       WORD 0x00 ; Ячейка для хранения разряда единиц координаты X
X2:     WORD 0x00 ; Ячейка, которая хранит 2*X_COORD

KILLS:      WORD 0x00 ; Счётчик попаданий, изначально сброшен в 0

RELOAD:     LD $ADR1
        ST $PTR
        LD #0xB
        ST $CNT

; ---------------------------------- Процесс рисования зайца ----------------------------------

DRAW:       CLA
        IN 0x11 ; Заносим SR -> AC
        AND #0x40 ; Проверяем 6-ой бит SR
        BEQ DRAW ; Если Z==1, то дальше ждём готовности
```

```
        LD (PTR)+ ; Косвенная автоинкрементная загрузка значения по адресу, который лежит в PTR
        OUT 0x10 ; Выводим значение AC в бегущую строку

        LD $CURRENT_FLAG ; Загружаем текущее значение счётчика
        INC
        ST $CURRENT_FLAG

        CMP $EXPECTED_FLAG
        BEQ PREPARING ; Если значение EXPECTED_FLAG и CURRENT_FLAG совпали, переходим к стрельбе

CONTINUE:LOOP $CNT ; Если CNT <= 0, то перескакиваем через одну команду

        JUMP $DRAW ; Если CNT > 0, переходим к рисованию следующего столбца зайца

        JUMP $RELOAD ; Если заяц нарисован, переходим к "перезарядке" значений для нового зайца

; ------------------------------------- Код ввода и парсинга координаты выстрела -------------------------------------

PREPARING:      SUB #0xB ; Вычитаем из AC (текущий счётчик) число 11
        ST $CURRENT_FLAG ; Сохраняем в ячейку с счётчиком

IN_X: CLA ; Затираем аккумулятор
        IN 0x19 ; Заносим SR ВУ-8 в AC
        AND #0x40 ; Проверяем 6-ой бит
        BEQ IN_X ; Если устройство не готово, ждём готовности дальше
        IN 0x18 ; Заносим DR -> AC, теперь в аккумуляторе кодировка введённого символа

        CMP $DELIMITER ; Выставляем признаки результата вычитания разделяющего символа x
        BEQ IN_Y ; Если был введён символ x, переходим к вводу координаты Y

        SUB #0x30 ; получаем введённую цифру, вычитая из её кода значение 0x30
        ST $TEMP ; Сохраняем цифру в ячейку TEMP

        LD $X_COORD ; Загружаем содержимое X_COORD в AC
        ASL ; Умножаем AC на 2
        ST $X2 ; Сохраняем удвоенную координату X
        ASL ; Умножаем AC на 2
```

```
        ASL ; Умножаем AC на 2
        ADD $X2 ; Прибавляем 2*X к AC (получаем 8*X + 2*X  = 10*X)
        ADD $TEMP ; Прибавляем разряд единиц для X
        ST $X_COORD ; Сохраняем результат в X_COORD

        JUMP IN_X ; Переходим ко вводу следующего символа (может быть символ x или цифра для разряда единиц у координаты)

IN_Y: CLA ; Чистим AC
        IN 0x19 ; Заносим SR ВУ-8 в AC
        AND #0x40 ; Проверяем 6-ой бит
        BEQ IN_Y ; Если устройство не готово, ждём готовности дальше
        IN 0x18 ; Заносим DR -> AC, теперь в аккумуляторе кодировка введённого символа

        SUB #0x30 ; Получаем введённую цифру, вычитая из её кода значение 0x30
        SUB #0x01 ; Вычитаем заранее единицу, чтобы Y лежал в диапазоне от 0 до 7 (для попаданий нужно)
        ST $Y_COORD ; Сохраняем координату Y в ячейку Y_COORD

; ------------------------------------- Код обнаружения попаданий по зайцу -------------------------------------

X_CHECK:  LD $X_COORD ; Загружаем в AC координату X
        CMP #10 ; Сравниваем со столбцом 10
        BLT GET_INDEX1 ; Если X < 10 переходим к получению индекса
        BEQ INDICATOR ; Если Z==1, переходим к выводу текущего числа попаданий без проверки Y

        CMP #11 ; Сравниваем со столбцом 11
        BEQ INDICATOR ; Если Z==1, переходим к выводу текущего числа попаданий без проверки Y

        CMP #22 ; Сравниваем со столбцом 22
        BLT GET_INDEX2 ; Если 11 < X < 22, переходим к получению индекса
        BEQ INDICATOR ; Если Z==1, переходим к выводу текущего числа попаданий без проверки Y

        CMP #32 ; Сравниваем со столбцом 32
        BLT GET_INDEX3 ; Если 22 < X < 32, переходим к получению индекса
        BEQ INDICATOR ; Если Z==1, переходим к выводу текущего числа попаданий без проверки Y

GET_INDEX1:      SUB #0x01 ; Вычитаем единичку для диапазона от 0 до 8
```

```
        ADD $ADR1 ; Прибавляем адрес начала массива столбцов зайца для получения конкретного столбца
        ST $X_INDEX ; Сохраняем в X_INDEX адрес, в котором расположен столбец, в который стреляем
        JUMP Y_CHECK ; Переходим к проверке Y

GET_INDEX2:     CMP #21 ; Сравниваем со столбцом 21
        BEQ INDICATOR ; Если Z==1, переходим к выводу текущего числа попаданий без проверки Y
        SUB #0x0C ; Вычитаем 12 из AC (сюда включено вычитание единички для индекса)
        ADD $ADR1 ; Прибавляем адрес начала массива столбцов зайца для получения конкретного столбца
        ST $X_INDEX
        JUMP Y_CHECK ; Переходим к проверке Y

GET_INDEX3:     SUB #0x17 ; Вычитаем 23 из AC (сюда опять включено вычитание единицы для индекса)
        ADD $ADR1 ; Прибавляем адрес начала массива столбцов зайца для получения конкретного столбца
        ST $X_INDEX
        JUMP Y_CHECK ; Переходим к проверке Y

Y_CHECK:  LD  $HIT_ADR1 ; Загружаем в AC адрес, по которому хранится нижний квадратик бегущей строки
        ADD $Y_COORD ; Добавляем к этому адресу смещение в виде координаты Y
        ST $Y_INDEX ; Сохраняем адрес ячейки, в которой хранится нужный нам элемент массива HIT_BOX
        LD (Y_INDEX) ; Загружаем содержимое ячейки с адресом, который лежит в Y_INDEX

        AND (X_INDEX) ; Логическое побитовое & с прямой косвенной адресацией, сравниваем столбец зайца с координатой Y
        BEQ INDICATOR ; Если получили 0 (Z==1), сразу переходим к выводу прошлого числа попаданий

        LD $KILLS ; Загружаем в AC счётчик попаданий
        INC ; Инкрементируем аккумулятор
        ST $KILLS ; Загружаем количество попаданий в ячейку KILLS

; ------------------------------------ Вывод количества попаданий по зайцам ------------------------------------

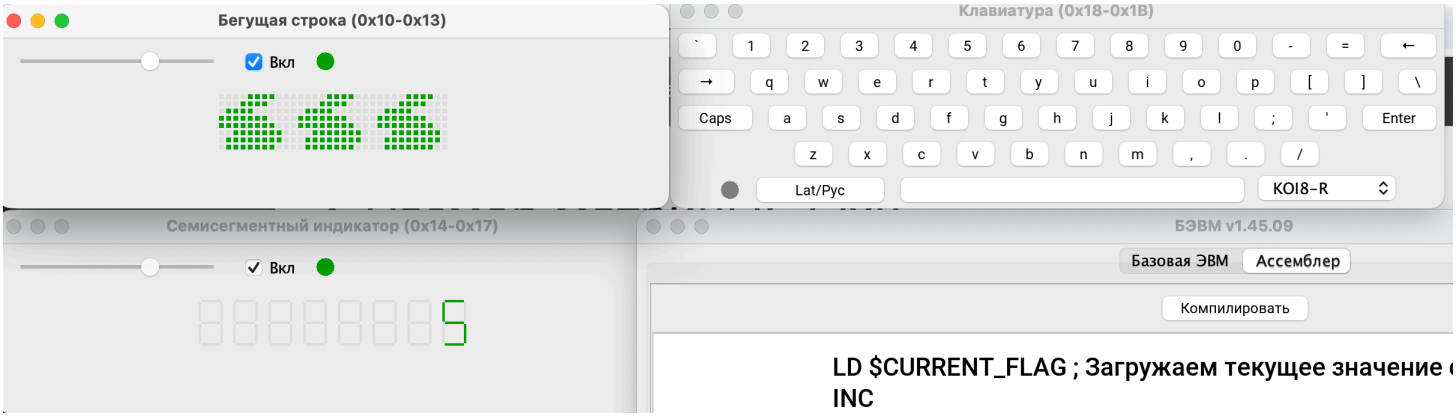
INDICATOR:      IN 0x15 ; Заносим SR семисегментного индикатора в AC
```

```
AND #0x40 ; Проверяем 6-ой бит
BEQ INDICATOR ; Если Z==1, дальше ожидаем готовности

LD $KILLS ; Загружаем количество попаданий в аккумулятор
OUT 0x14 ; Выводим количество попаданий

CLA ; Очищаем аккумулятор, для загрузки 0x00 в X_COORD
ST $X_COORD ; Обнуляем X_COORD

JUMP CONTINUE ; Переходим к продолжению рисования наших зайцев
```





Вывод

В ходе выполнения лабораторной работы №5 я ознакомился с асинхронным вводом и выводом данных в БЭВМ, узнал о различных внешних устройствах, о контроллерах, изучил написание программы на языке ассемблера. Я научился работать с вводом данных при помощи ВУ-2, узнал об их представлении в различных кодировках, а также прочитал о различных способах обмена данными, их передачей и приёмом.